



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems

Lecture 13

Scheduling: deadlocks

Prof. Yunming XIAO
School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Real-time scheduling

Real-time scheduling

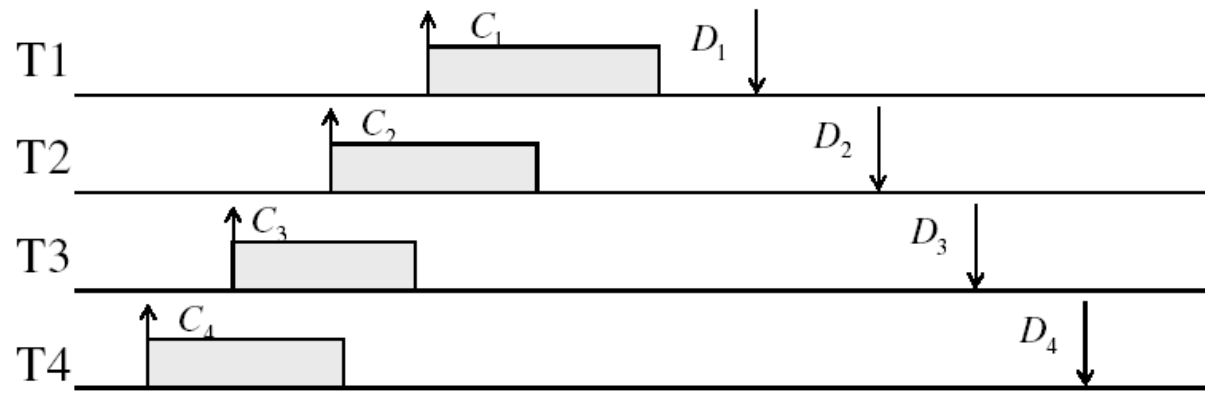
- Goal: **predictability** of performance!
 - We need to predict with confidence worst case response times for systems!
 - In RTS, performance guarantees are:
 - Task- and/or class centric and often ensured a priori
 - In conventional systems, performance is:
 - System/throughput oriented with post-processing (... wait and see ...)
 - Real-time is about enforcing predictability, and does not equal fast computing!!!

Real-time scheduling

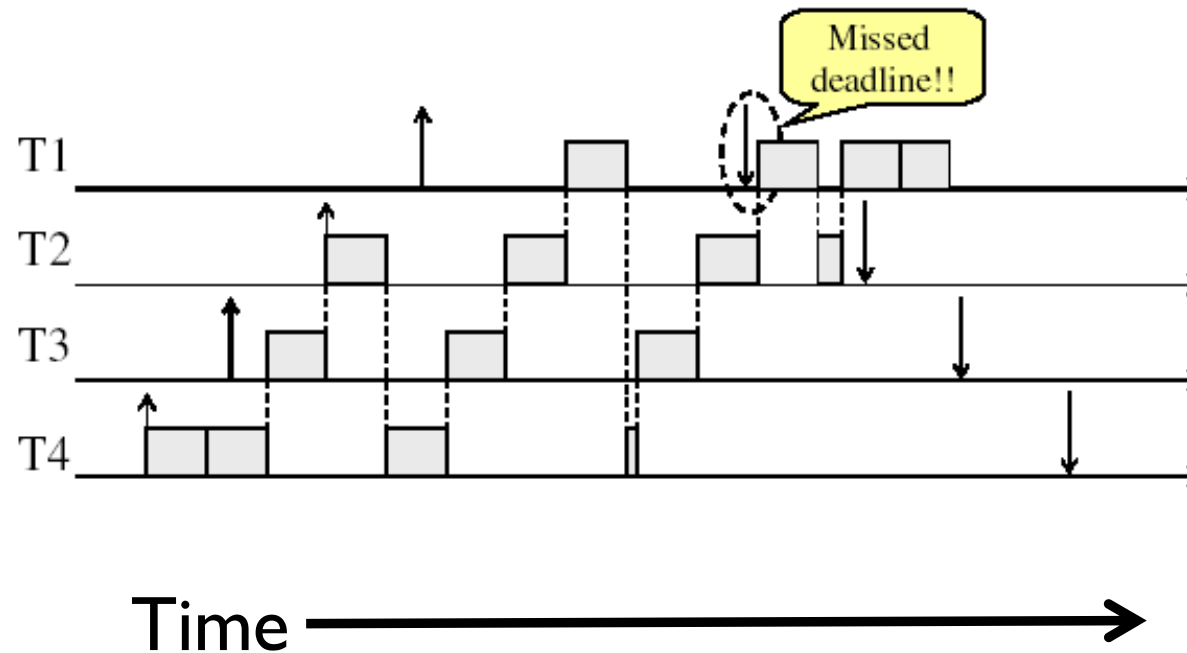
- Hard real-time: for time-critical safety-oriented systems
 - Meet all deadlines (if at all possible)
 - Ideally: determine in advance if this is possible
 - Earliest Deadline First (EDF), Least Laxity First (LLF), Rate-Monotonic Scheduling (RMS), Deadline Monotonic Scheduling (DM)
- Soft real-time: for multimedia
 - Attempt to meet deadlines with high probability
 - Constant Bandwidth Server (CBS)

Example: workload characteristics

- Tasks are preemptable, independent with arbitrary arrival (=release) times
- Tasks have deadlines (D) and known computation times (C)
- Example Setup:

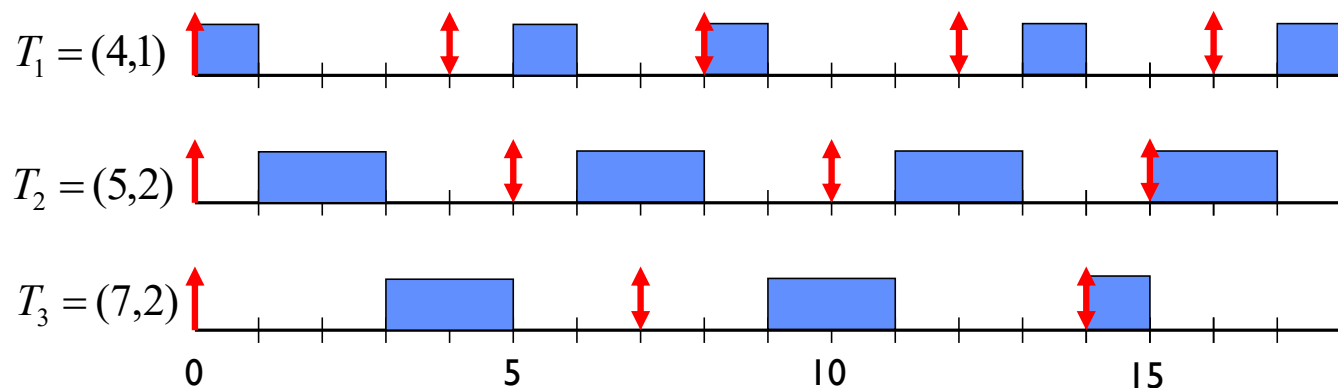


Example: Round-Robin scheduling doesn't work



Earliest Deadline First (EDF)

- Tasks **periodic** with period P and computation C in each period: (P_i, C_i) for each task i
- Preemptive priority-based dynamic scheduling:
 - Each task is assigned a (current) priority based on how close the absolute deadline is (i.e. $D_i^{t+1} = D_i^t + P_i$ for each task!)
 - **The scheduler always schedules the active task with the closest absolute deadline**



EDF feasibility testing

- Even EDF won't work if you have too many tasks
- For n tasks with computation time C and deadline D , a feasible schedule exists if:

$$\sum_{i=1}^n \left(\frac{C_i}{D_i} \right) \leq 1$$

Multi-core scheduling

Multi-core scheduling

- Algorithmically, not a huge difference from single-core scheduling
- Implementation-wise, helpful to have *per-core* scheduling data structures
 - Cache coherence
- *Affinity scheduling*: once a thread is scheduled on a CPU, OS tries to reschedule it on the same CPU
 - Cache reuse, branch prediction
 - Example for $O(1)$ scheduler: one set of queues/core with background rebalancing

Spinlocks for multiprocessing

- Spinlock implementation:

```
int value = 0; // Free
Acquire() {
    while (test&set(&value)) {}; // spin while busy
}
Release() {
    value = 0; // atomic store
}
```

- Spinlock doesn't put the calling thread to sleep—it just busy waits
 - When might this be preferable?
 - Waiting for limited number of threads at a barrier in a multiprocessing (multicore) program
 - Wait time at barrier would be greatly increased if threads must be woken inside kernel
- Every test&set() atomic read & write
 - Makes value ping-pong around between core-local caches (using lots of memory!)

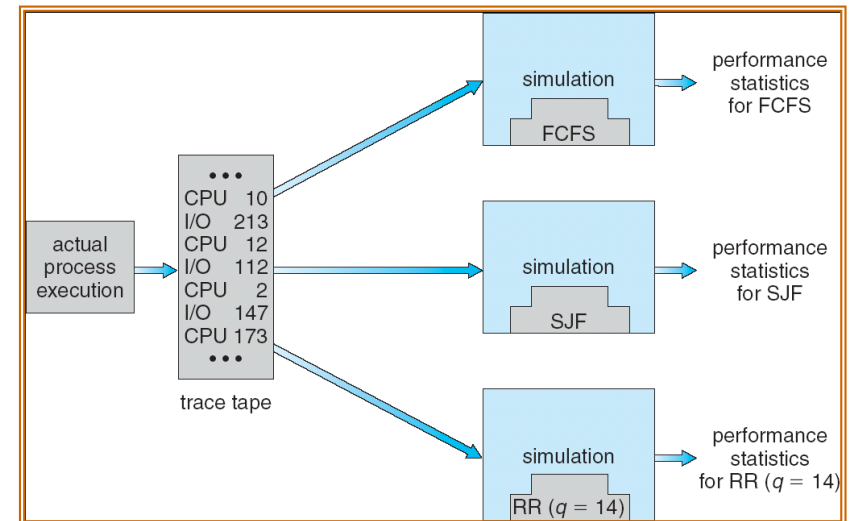
Spinlocks for multiprocessing (cont'd)

- Want test&test&set() !
 - First test just reads the lock; no memory contention!
 - Only if free do atomic test&set()
- The extra read eliminates the ping-ponging issues:

```
// Implementation of test&test&set():
Acquire() {
    do {
        while(value);           // wait until might be free
    } while (test&set(&value)); // exit if acquire lock
}
```

How to evaluate a scheduling algorithm?

- Deterministic modeling
 - takes a predetermined workload and compute the performance of each algorithm for that workload
- Queueing models
 - Mathematical approach for handling stochastic workloads
- Implementation/Simulation:
 - Build system which allows actual algorithms to be run against actual data
 - Most flexible/general

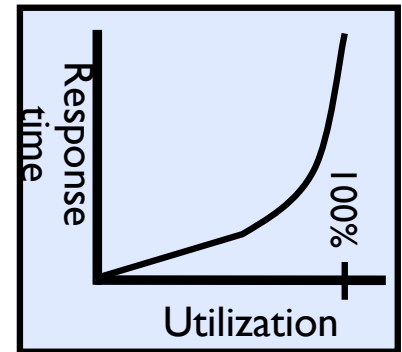


Choosing the right scheduler

I Care About:	Then Choose:
CPU Throughput	FCFS
Avg. Response Time	SRTF Approximation
I/O Throughput	SRTF Approximation
Fairness (CPU Time)	Round Robin, Fair Sharing, Lottery, EEVDF, ...
Meeting Deadlines	EDF
Favoring Important Tasks	Priority

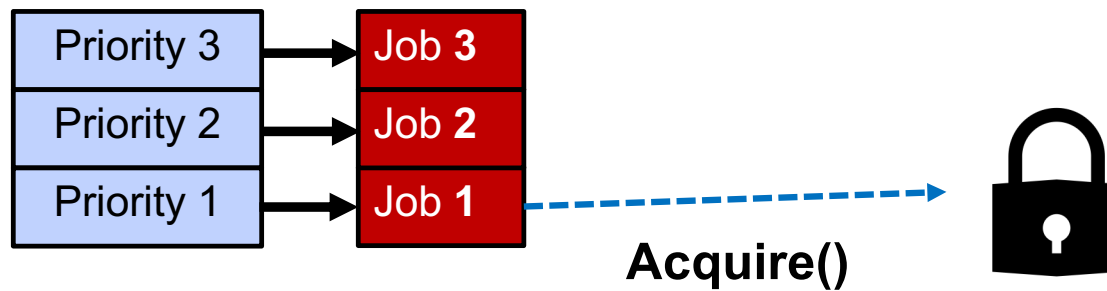
A final word on scheduling

- When do the details of the scheduling policy and fairness really matter?
 - When there aren't enough resources to go around
- When should you simply buy a faster computer?
 - (Or network link, or expanded highway, or ...)
 - One approach: Buy it when it will pay for itself in improved response time
 - Perhaps you're paying for worse response time in reduced productivity, customer angst, etc...
 - Might think that you should buy a faster X when X is utilized 100%, but usually, response time goes to infinity as utilization $\Rightarrow$ 100%
- An interesting implication of this curve:
 - Most scheduling algorithms work fine in the "linear" portion of the load curve, fail otherwise
 - Argues for buying a faster X when hit "knee" of curve



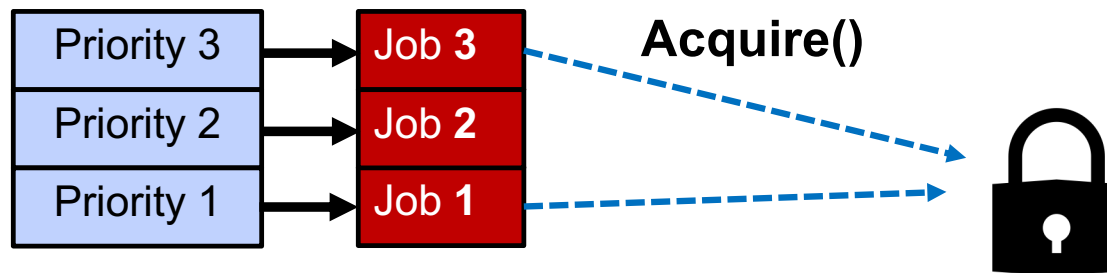
Priority inversion

Priority inversion



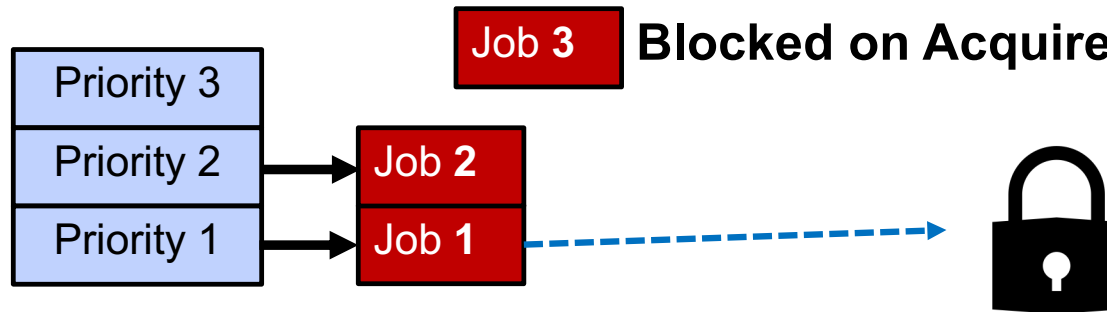
- **At this point, which job does the scheduler choose?**
- Job 3 (Highest priority)

Priority inversion



- Job 3 attempts to acquire lock held by Job 1

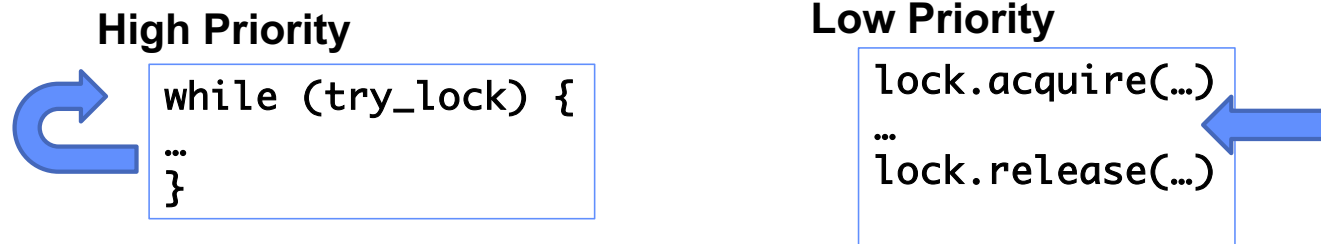
Priority inversion



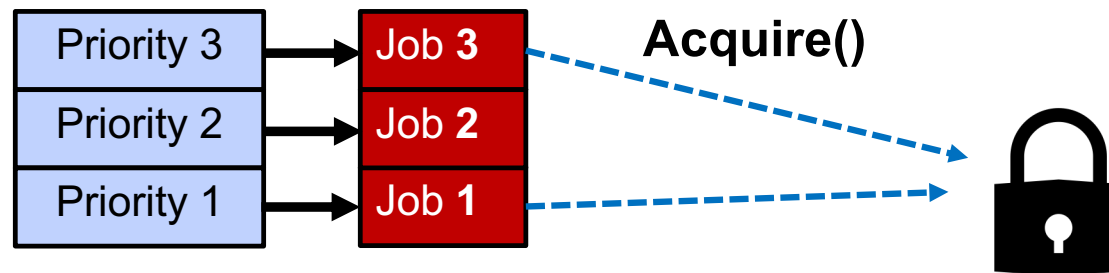
- **At this point, which job does the scheduler choose?**
- Job 2 (Medium Priority)
- **Priority inversion**

Priority inversion

- Where high priority task is blocked waiting on low priority task
- Low priority one ***must*** run for high priority to make progress
- Medium priority task can starve a high priority one
- Example:

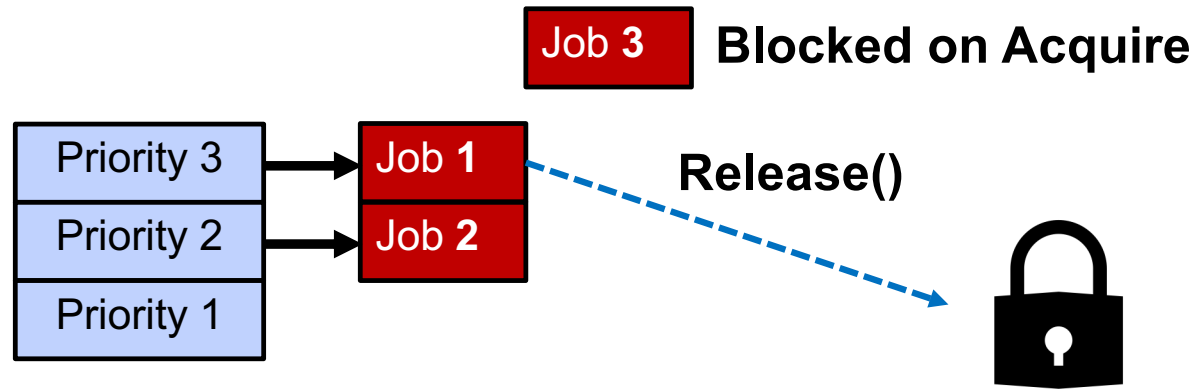


One solution: priority donation/inheritance



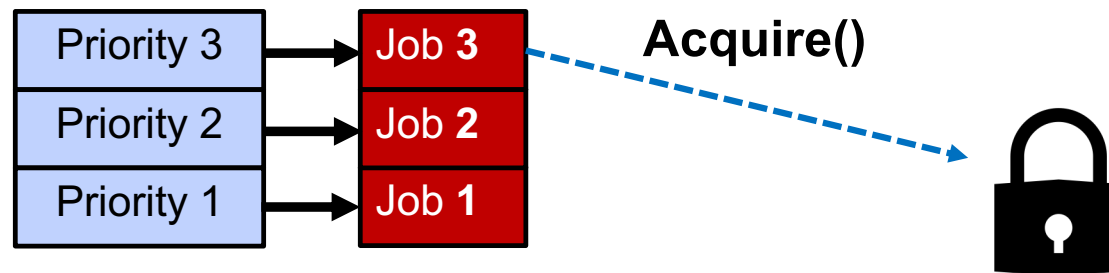
- Job 3 temporarily grants Job 1 its “high priority” to run on its behalf

One solution: priority donation/inheritance



- Job 3 temporarily grants Job 1 its “high priority” to run on its behalf

One solution: priority donation/inheritance



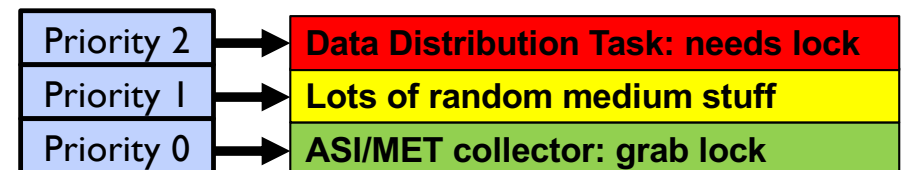
- Job 1 completes critical section and releases lock
- Job 3 acquires lock, runs again
- How does the scheduler know?

Case study: Martian Pathfinder Rover

- July 4, 1997 – Pathfinder lands on Mars
 - First US Mars landing since Vikings in 1976; first rover
- And then...a few days into mission...:
 - Multiple system resets occur to realtime OS (VxWorks)
 - System would reboot randomly, losing valuable time and progress



<https://youtu.be/3E9BcdFYENg?t=122>

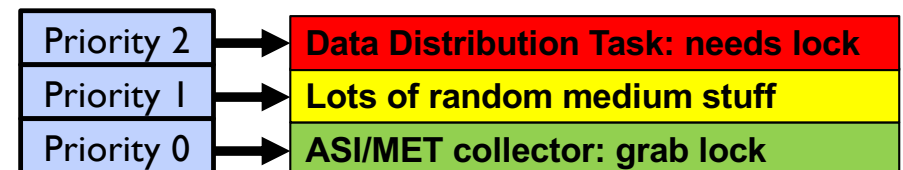


Case study: Martian Pathfinder Rover

- Problem? Priority Inversion!
 - Low priority task grabs mutex trying to communicate with high priority task:
 - Realtime watchdog detected lack of forward progress and invoked reset to safe state
 - High-priority data distribution task was supposed to complete with regular deadline
- Solution: Turn priority donation back on and upload fixes!
- Original developers turned off priority donation (also called priority inheritance)
 - Worried about performance costs of donating priority!



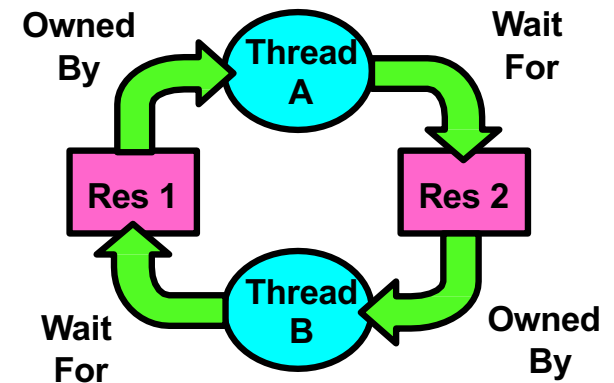
<https://youtu.be/3E9BcdFYENg?t=122>



Deadlocks

Deadlock: A deadly type of starvation

- Starvation: thread waits indefinitely
 - Example, low-priority thread waiting for resources constantly in use by high-priority threads
- Deadlock: circular waiting for resources
 - Thread A owns Res 1 and is waiting for Res 2; Thread B owns Res 2 and is waiting for Res 1
- Deadlock $\Rightarrow$ starvation but not vice versa
 - Starvation can end (but doesn't have to)
 - Deadlock can't end without external intervention



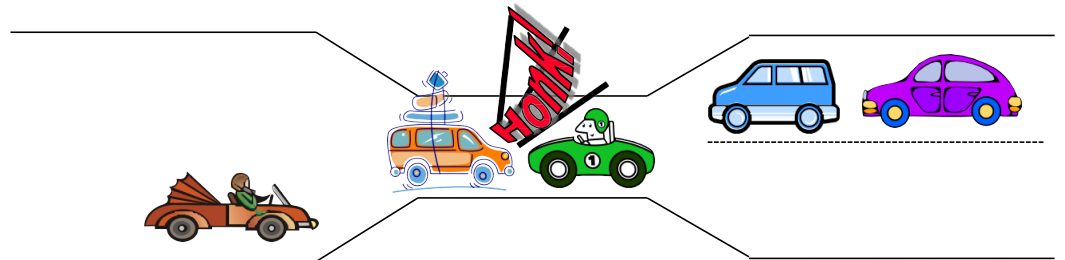
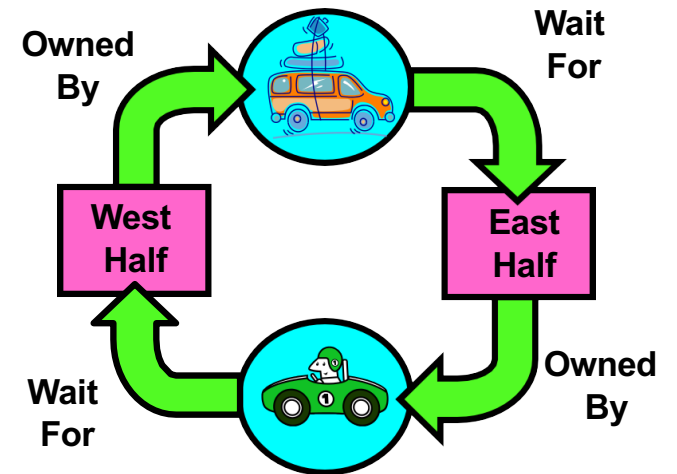
Example: single-lane bridge crossing



CA 140 to Yosemite National Park

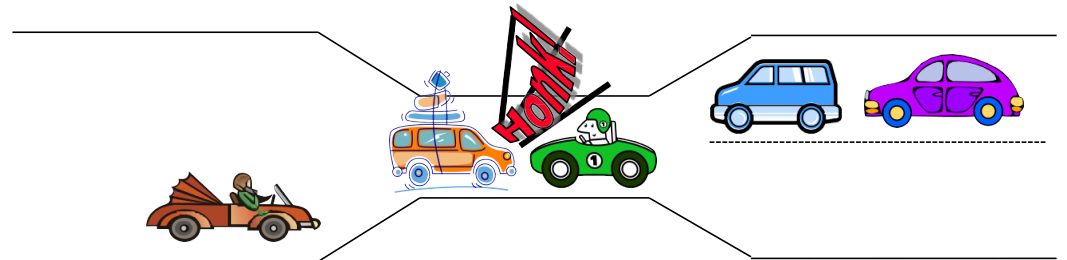
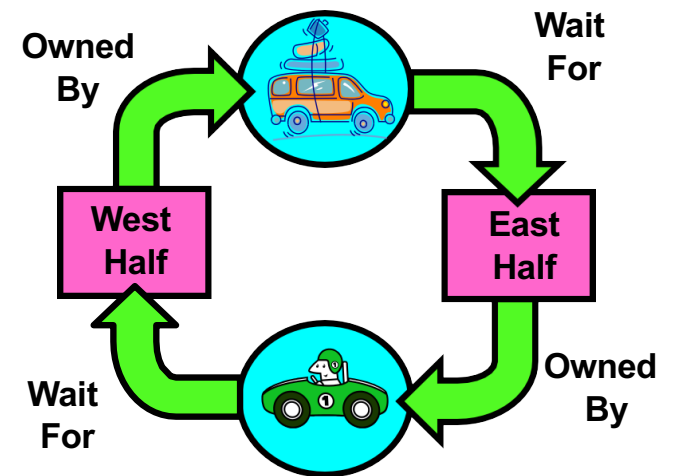
Bridge crossing example

- Each segment of road can be viewed as a resource
 - Car must own the segment under them
 - Must acquire segment that they are moving into
- For bridge: must acquire both halves
 - Traffic only in one direction at a time



Bridge crossing example

- **Deadlock**: shown on the right when two cars in opposite directions meet in middle
 - Each acquires one segment and needs next
 - Deadlock resolved if one car backs up (preempt resources and rollback)
 - Several cars may have to be backed up
- Starvation (not deadlock):
 - East-going traffic really fast $\Rightarrow$ no one gets to go west



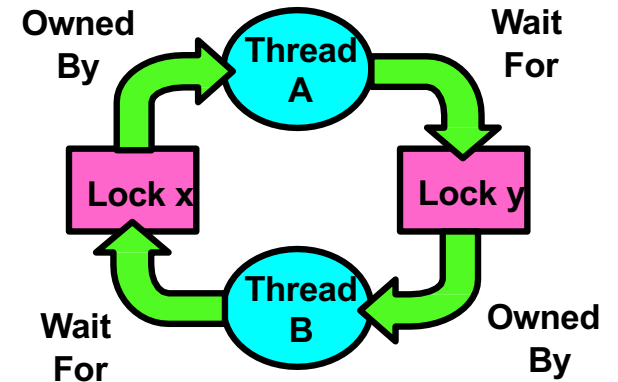
Deadlock with locks

Thread A:

```
x.Acquire();  
y.Acquire();  
...  
y.Release();  
x.Release();
```

Thread B:

```
y.Acquire();  
x.Acquire();  
...  
x.Release();  
y.Release();
```



- This lock pattern exhibits non-deterministic deadlock
 - Sometimes it happens, sometimes it doesn't!
- This is really hard to debug!

Deadlock with locks: “unlucky” case

Thread A:

x.Acquire();

y.Acquire(); *<stalled>*

<unreachable>

...

y.Release();

x.Release();

Thread B:

y.Acquire();

x.Acquire(); *<stalled>*

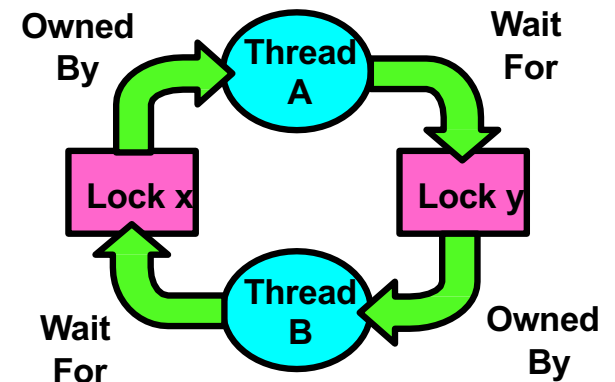
<unreachable>

...

x.Release();

y.Release();

- Neither thread will get to run $\Rightarrow$ deadlock



Deadlock with locks: “lucky” case

Thread A:

```
x.Acquire();  
y.Acquire();  
...  
y.Release();  
x.Release();
```

Thread B:

```
y.Acquire();  
  
x.Acquire();  
...  
x.Release();  
y.Release();
```

- Sometimes, schedule won't trigger deadlock!

Other types of deadlock

- Threads often block waiting for resources
 - Locks
 - Terminals
 - Printers
 - CD drives
 - Memory
- Threads often block waiting for other threads
 - Pipes
 - Sockets
- You can deadlock on any of these!

Deadlock with space

Thread A:

AllocateOrWait(1 MB)
AllocateOrWait(1 MB)
Free(1 MB)
Free(1 MB)

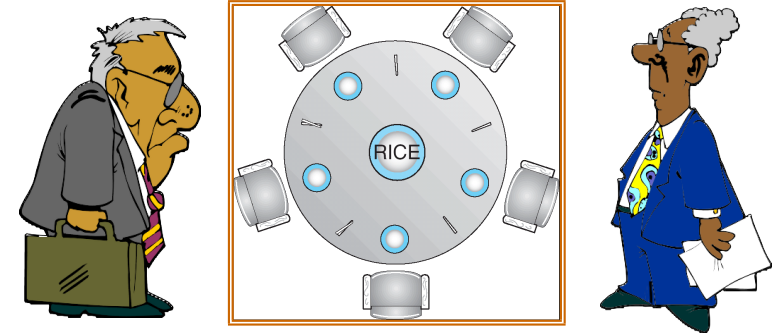
Thread B

AllocateOrWait(1 MB)
AllocateOrWait(1 MB)
Free(1 MB)
Free(1 MB)

- If only 2 MB of space, we get same deadlock situation

Dining lawyers problem

- Five chopsticks/Five lawyers (really cheap restaurant)
 - Free-for all: Lawyer will grab any one they can
 - Need two chopsticks to eat
- What if all grab at same time?
 - Deadlock!
- How to fix deadlock?
 - Make one of them give up a chopstick (Hah!)
 - Eventually everyone will get chance to eat
- How to prevent deadlock?
 - Never let lawyer take last chopstick if no hungry lawyer has two chopsticks afterwards
 - Can we formalize this requirement somehow?



Four requirements for occurrence of deadlock

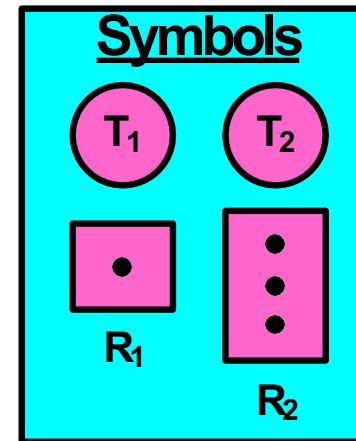
- **Mutual exclusion**
 - Only one thread at a time can use a resource.
- **Hold and wait**
 - Thread holding at least one resource is waiting to acquire additional resources held by other threads
- **No preemption**
 - Resources are released only voluntarily by the thread holding the resource, after thread is finished with it

Four requirements for occurrence of deadlock

- Circular wait
 - There exists a set $\{T_1, \dots, T_n\}$ of waiting threads
 - T_1 is waiting for a resource that is held by T_2
 - T_2 is waiting for a resource that is held by T_3
 - ...
 - T_n is waiting for a resource that is held by T_1

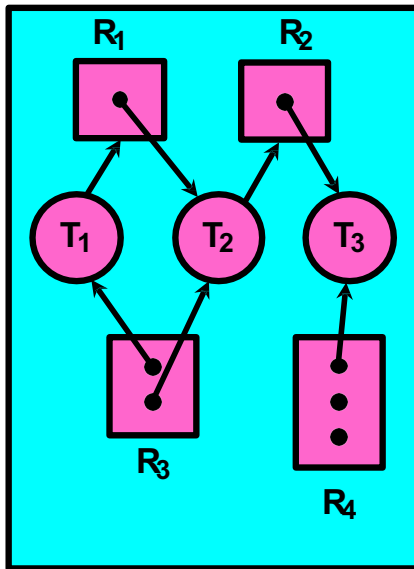
Detecting deadlock: resource-allocation graph

- System Model
 - A set of Threads $T_1, T_2, \dots, T_n$
 - Resource types $R_1, R_2, \dots, R_m$
 - CPU cycles, memory space, I/O devices
 - Each resource type R_i has W_i instances
 - Each thread utilizes a resource as follows:
 - Request() / Use() / Release()
- Resource-Allocation Graph:
 - V is partitioned into two types:
 - $T = \{T_1, T_2, \dots, T_n\}$, the set threads in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set of resource types in system

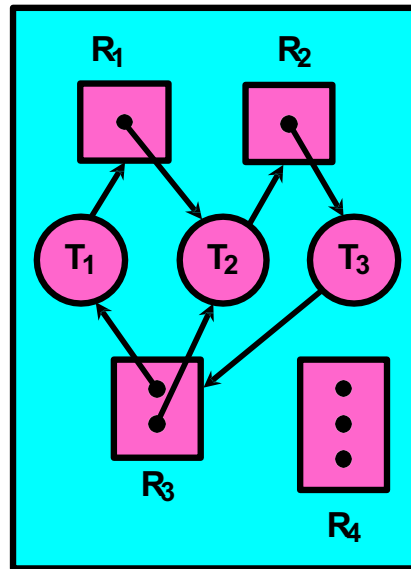


Resource-allocation graph examples

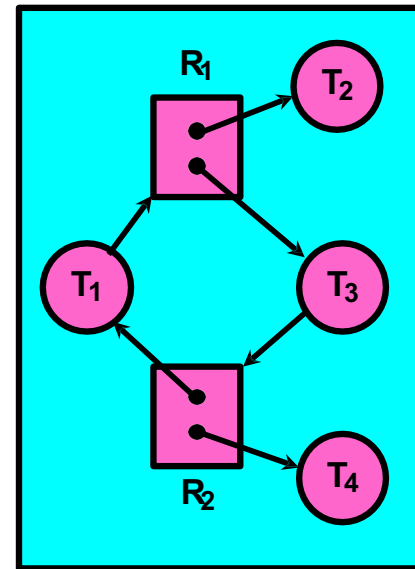
- Model:
 - request edge – directed edge $T_1 \rightarrow R_j$
 - assignment edge – directed edge $R_j \rightarrow T_i$



Simple Resource
Allocation Graph



Allocation Graph
With Deadlock



Allocation Graph With
Cycle, but No Deadlock

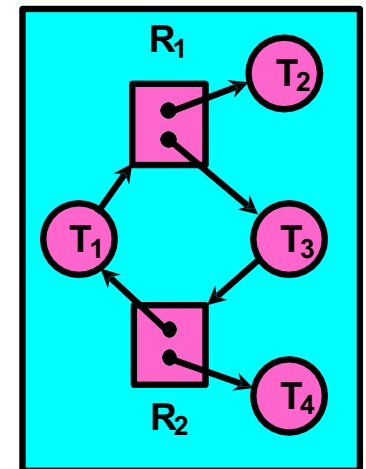
Deadlock detection algorithm

- Let $[X]$ represent an m -ary vector of non-negative integers (quantities of resources of each type):

$[FreeResources]$: Current free resources each type

$[Request_x]$: Current requests from thread X

$[Alloc_x]$: Current resources held by thread X



Deadlock detection algorithm

- See if tasks can eventually terminate on their own

[Avail] = [FreeResources]

Add all nodes to UNFINISHED

do {

 done = true

 Foreach node in UNFINISHED {

 if ([Request_{node}] <= [Avail]) {

 remove node from UNFINISHED

 [Avail] = [Avail] + [Alloc_{node}]

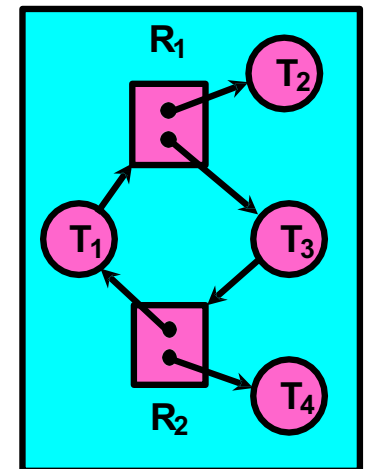
 done = false

 }

 }

} until(done)

- Nodes left in UNFINISHED $\Rightarrow$ deadlocked



How should a system deal with deadlock?

- Four different approaches:
 1. Deadlock prevention: write your code in a way that it isn't prone to deadlock
 2. Deadlock recovery: let deadlock happen, and then figure out how to recover from it
 3. Deadlock avoidance: dynamically delay resource requests so deadlock doesn't happen
 4. Deadlock denial: ignore the possibility of deadlock
- Modern operating systems:
 - Make sure the *system* isn't involved in any deadlock
 - Ignore deadlock in applications
 - “Ostrich Algorithm”

Techniques for preventing deadlock

- Infinite resources
 - Include enough resources so that no one ever runs out of resources.
Doesn't actually have to be infinite, just large...
 - Give illusion of infinite resources (e.g. virtual memory)
 - Examples:
 - Bay bridge with 12,000 lanes. Never wait!
 - Infinite disk space (not realistic yet?)

Techniques for preventing deadlock

- No sharing of resources (totally independent threads)
 - Not very realistic
- Don't allow waiting
 - How the phone company avoids deadlock
 - Call Mom in Toledo, works way through phone network, but if blocked get busy signal.
 - Technique used in Ethernet/some multiprocessor nets
 - Everyone speaks at once. On collision, back off and retry
- Inefficient, since have to keep retrying
 - Consider: driving to Macau; when hit traffic jam, suddenly you're transported back home and told to retry!

(Virtually) infinite resources

Thread A:

```
AllocateOrWait(1 MB)
AllocateOrWait(1 MB)
Free(1 MB)
Free(1 MB)
```

Thread B

```
AllocateOrWait(1 MB)
AllocateOrWait(1 MB)
Free(1 MB)
Free(1 MB)
```

- With virtual memory we have “infinite” space so everything will just succeed, thus above example won’t deadlock
 - Of course, it isn’t actually infinite, but certainly larger than 2MB!

Techniques for preventing deadlock

- Make all threads request everything they'll need at the beginning.
 - Problem: Predicting future is hard, tend to over-estimate resources
 - Example:
 - If need 2 chopsticks, request both at same time
 - Don't leave home until we know no one is using any intersection between here and where you want to go; only one car on the Hong Kong–Zhuhai–Macao Bridge at a time

Techniques for preventing deadlock

- Force all threads to request resources in a particular order preventing any cyclic use of resources
 - Thus, preventing deadlock
 - Example (x.Acquire(), y.Acquire(), z.Acquire(),...)
 - Make tasks request disk, then memory, then...
 - Keep from deadlock on freeways around SF by requiring everyone to go clockwise

Request resources atomically (1)

- Rather than:

Thread A:

```
x.Acquire();  
y.Acquire();  
...  
y.Release();  
x.Release();
```

Thread B:

```
y.Acquire();  
x.Acquire();  
...  
x.Release();  
y.Release();
```

- Consider instead:

Thread A:

```
Acquire_both(x, y);  
...  
y.Release();  
x.Release();
```

Thread B:

```
Acquire_both(y, x);  
...  
x.Release();  
y.Release();
```

Request resources atomically (2)

- Or consider this:

Thread A:

z.Acquire();

x.Acquire();

y.Acquire();

z.Release();

...

y.Release();

x.Release();

Thread B:

z.Acquire();

y.Acquire();

x.Acquire();

z.Release();

...

x.Release();

y.Release();

Acquire resources in consistent order

- Rather than:

Thread A:

```
x.Acquire();  
y.Acquire();  
...  
y.Release();  
x.Release();
```

Thread B:

```
y.Acquire();  
x.Acquire();  
...  
x.Release();  
y.Release();
```

- Consider instead:

Thread A:

```
x.Acquire();  
y.Acquire();  
...  
y.Release();  
x.Release();
```

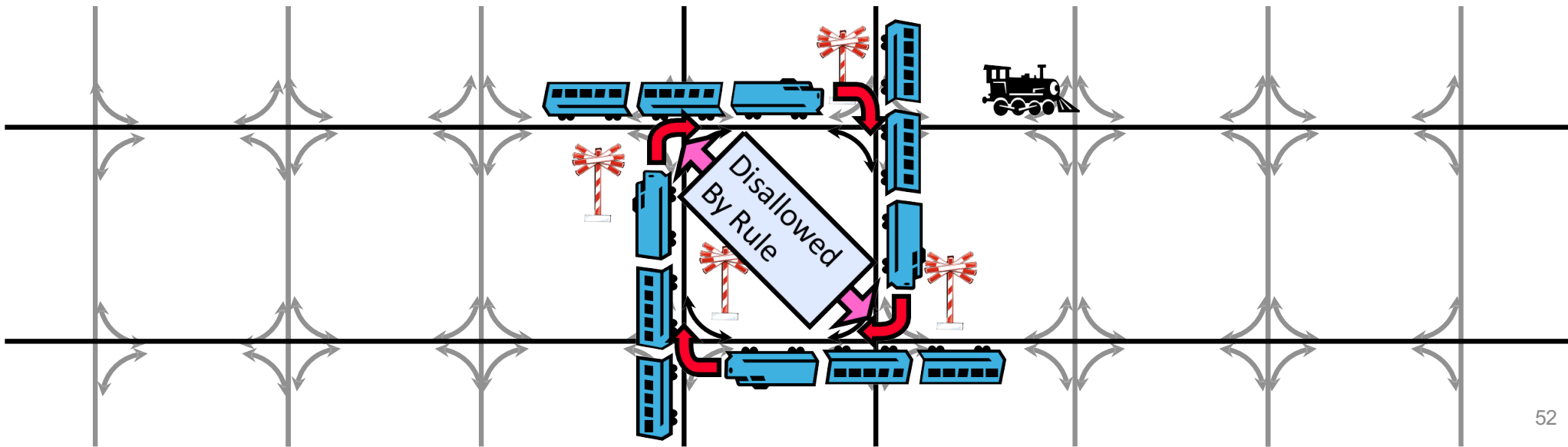
Thread B:

```
x.Acquire();  
y.Acquire();  
...  
x.Release();  
y.Release();
```

**Does it matter
in which order
the locks are
released?**

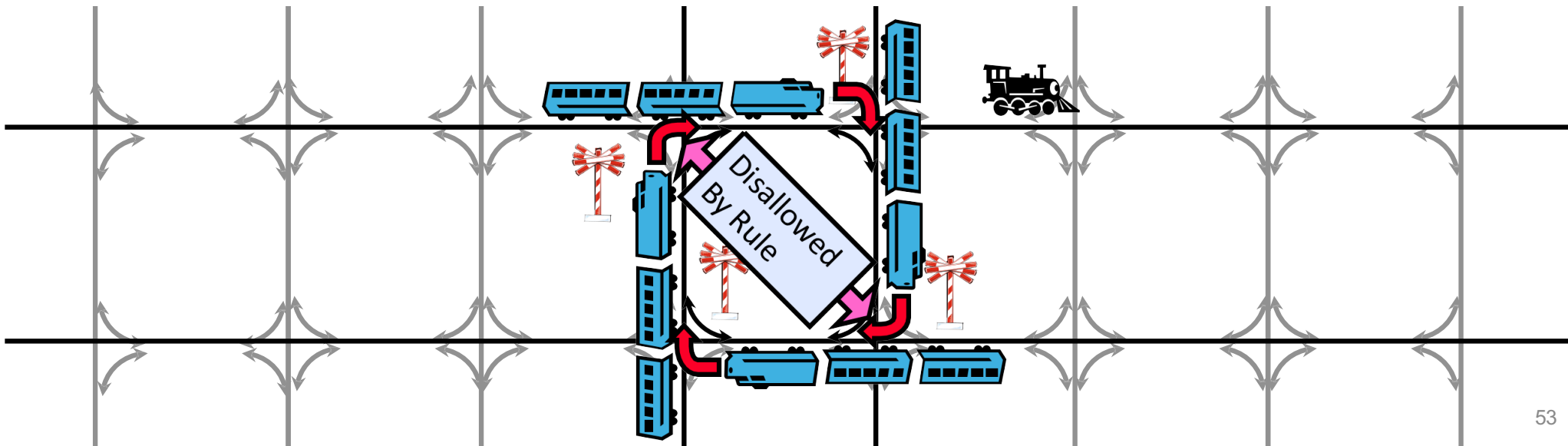
Train example (Wormhole-Routed network)

- Circular dependency (deadlock!)
 - Each train wants to turn right, but is blocked by other trains
- Similar problem to multiprocessor networks
 - Wormhole-Routed Network: Messages trail through network like a “worm”



Train example (Wormhole-Routed network)

- Fix? Imagine grid extends in all four directions
 - **Force ordering of channels** (tracks)
 - Protocol: Always go east-west first, then north-south
 - Called “dimension ordering” (X then Y)



Techniques for recovering from deadlock

- Terminate thread, force it to give up resources
 - In Bridge example, Godzilla picks up a car, hurls it into the river. Deadlock solved!
 - Hold dining lawyer in contempt and take away in handcuffs
 - But, not always possible – killing a thread holding a mutex leaves world inconsistent
- Preempt resources without killing off thread
 - Take away resources from thread temporarily
 - Doesn't always fit with semantics of computation

Techniques for recovering from deadlock

- Roll back actions of deadlocked threads
 - For bridge example, make one car roll backwards (may require others behind him)
 - Common technique in databases (transactions)
 - Of course, if you restart in exactly same way, may reenter deadlock once again
- Many operating systems use other options

Another view of virtual memory: pre-empting resources

Thread A:

AllocateOrWait(1 MB)
AllocateOrWait(1 MB)
Free(1 MB)
Free(1 MB)

Thread B

AllocateOrWait(1 MB)
AllocateOrWait(1 MB)
Free(1 MB)
Free(1 MB)

- Before: With virtual memory we have “infinite” space so everything will just succeed, thus above example won’t deadlock
 - Of course, it isn’t actually infinite, but certainly larger than 2MB!
- Alternative view: we are “pre-empting” memory when paging out to disk, and giving it back when paging back in
 - This works because thread can’t use memory when paged out

Techniques for deadlock avoidance

- Idea: When a thread requests a resource, OS checks if it would result in a deadlock
 - If not, it grants the resource right away
 - If so, it waits for other threads to release resources

THIS DOES NOT WORK!!!!

- Example:

	<u>Thread A:</u>	<u>Thread B:</u>	
	x.Acquire();	y.Acquire();	
Blocks...	y.Acquire();	x.Acquire();	
	...	...	Wait?
	y.Release();	x.Release();	But it's already too late...
	x.Release();	y.Release();	

Deadlock avoidance: three states

- Safe state
 - System can delay resource acquisition to prevent deadlock
- Unsafe state
 - No deadlock yet...
 - But it is unavoidable
 - All possible patterns that threads can request resources lead to deadlock!
- Deadlocked state
 - There exists a deadlock in the system
 - Also considered “unsafe”

Deadlock avoidance: prevent system from reaching an *unsafe* state

Deadlock avoidance

- Idea: When a thread requests a resource, OS checks if it would result in ~~deadlock~~ **an unsafe state**
 - If not, it grants the resource right away
 - If so, it waits for other threads to release resources
- Example:

Thread A:

```
x.Acquire();  
y.Acquire();  
...  
y.Release();  
x.Release();
```

Thread B:

```
y.Acquire();  
x.Acquire();  
...  
x.Release();  
y.Release();
```

**Wait until
Thread A
releases
mutex X**

Banker's Algorithm for avoiding deadlock



- Toward right idea:
 - State maximum (max) resource needs in advance
 - Allow particular thread to proceed if:
 - $(\text{available resources} - \# \text{requested}) \geq \text{max remaining that might be needed by any thread}$
- Banker's algorithm (less conservative):
 - Allocate resources dynamically
 - Evaluate each request and grant if some ordering of threads is still deadlock free afterward
 - Technique: pretend each request is granted, then run deadlock detection algorithm, substituting:
$$([\text{Max}_{\text{node}}] - [\text{Alloc}_{\text{node}}] \leq [\text{Avail}]) \text{ for } ([\text{Request}_{\text{node}}] \leq [\text{Avail}])$$
 - Grant request if result is deadlock free (conservative!)

Banker's Algorithm for avoiding deadlock

```
[Avail] = [FreeResources]
Add all nodes to UNFINISHED
do {
  done = true
  Foreach node in UNFINISHED {
    if ([Requestnode] <= [Avail]) {
      remove node from UNFINISHED
      [Avail] = [Avail] + [Allocnode]
      done = false
    }
  }
} until(done)
```



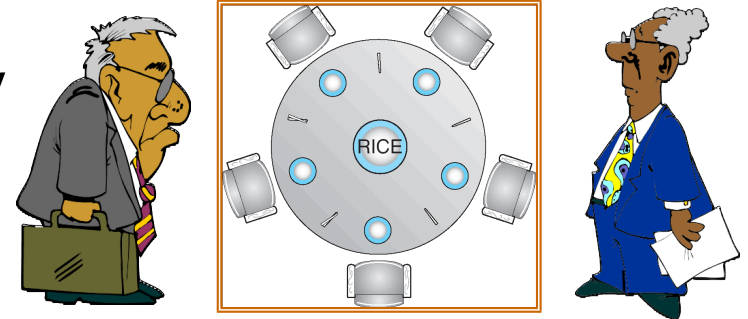
Banker's Algorithm for avoiding deadlock

```
[Avail] = [FreeResources]
Add all nodes to UNFINISHED
do {
  done = true
  Foreach node in UNFINISHED {
    if ( $[Max_{node}] - [Alloc_{node}] \leq [Avail]$ ) {
      remove node from UNFINISHED
       $[Avail] = [Avail] + [Alloc_{node}]$ 
      done = false
    }
  }
} until(done)
```



Banker's Algorithm Example

- Banker's algorithm with dining lawyers
 - “Safe” (won't cause deadlock) if when try to grab chopstick either:
 - Not last chopstick
 - Is last chopstick but someone will have two afterwards
- What if k-handed lawyers? Don't allow if:
 - It's the last one, no one would have k
 - It's 2nd to last, and no one would have k-1
 - It's 3rd to last, and no one would have k-2
 - ...



Conclusion

- Four conditions for deadlocks
 - Mutual exclusion
 - Hold and wait
 - No preemption
 - Circular wait

Conclusion

- Techniques for addressing Deadlock
 - Deadlock prevention:
 - write your code in a way that it isn't prone to deadlock
 - Deadlock recovery:
 - let deadlock happen, and then figure out how to recover from it
 - Deadlock avoidance:
 - dynamically delay resource requests so deadlock doesn't happen
 - Banker's Algorithm provides an algorithmic way to do this
 - Deadlock denial:
 - ignore the possibility of deadlock

Reading for next lecture

- See papers available on blackboard

Thanks